

1 Problem setting

A scalar advection-diffusion problem without reaction.

$$u_t + \nabla \cdot (F(u) - D(u)\nabla u) = 0, \mathbf{x} \in \mathbb{R}^d \quad (1)$$

The variable $D(u)$ will be treated with Kirchhoff transformation. The diffusion coefficient D is treated as a constant here. This will not affect the following discussion on numerical scheme.

$$u_t + \nabla \cdot F(u) - D \Delta u = 0, \mathbf{x} \in \mathbb{R}^d \quad (2)$$

1.1 Finite volume scheme

Replace scalar u with cell averaged value

$$\bar{u} = \frac{1}{|E|} \int_E u(\mathbf{x}, t) d\mathbf{x} \quad (3)$$

and Lax-Friedrichs numerical flux

$$f(u) = \frac{1}{2} [F(u^+) + F(u^-) \cdot \mathbf{v}_E - \alpha(u^+ - u^-)] \quad (4)$$

The finite volume scheme of advection-diffusion problem is:

$$\bar{u}_t + \frac{1}{|E|} \int_{\partial E} (\hat{f}(\bar{u}) - \nabla \bar{u} \cdot \mathbf{v}_E) d\sigma(\mathbf{x}) = 0 \quad (5)$$

In 2D quadrilateral logically rectangular mesh, we apply gaussian quadrature every edge as the approximation of integral of flux. I use Multi-level WENO reconstruction scheme to reconstruct the point value in advective numerical flux.

$$\hat{f}(u) = \frac{1}{2} [F(\mathcal{R}^+(\{\bar{u}\}_+)) + F(\mathcal{R}^-(\{\bar{u}\}_-)) \cdot \mathbf{v}_E - \alpha(\mathcal{R}^+(\{\bar{u}\}_+) - \mathcal{R}^-(\{\bar{u}\}_-))] \quad (6)$$

Let $\{\bar{u}\}_+ = \{\bar{u}_i \mid i \in \text{WENO stencil } I_+\}$. Diffusive flux is different from its advective counterpart. I reconstruct derivative across the edge by sampling along the normal direction.

2 Computation of Jacobian

Implicit method contains solving a non linear system and a linear subproblem. I write the general non linear system as:

$$\bar{u}_t + \mathcal{F}(\bar{u}) - \mathcal{G}(\bar{u}) = 0 \quad (7)$$

where $\mathcal{F}(\bar{u})$ represents advection part, and $\mathcal{G}(\bar{u})$ represents diffusion part. The semi discretized form is

$$\bar{u}^{n+1} - \bar{u}^n + \Delta t [\mathcal{F}(\bar{u}^{n+1}) - \mathcal{G}(\bar{u}^{n+1})] = 0 \quad (8)$$

There are many well-researched numerical methods that can be used to solve such non linear system.

2.1 ML-WENO reconstruction

I use ML-WENO reconstruction scheme to reconstruct point values from cell-averaged value... I use two stage non linear weighting. The first stage is calculated as

$$\check{\omega}_l = \frac{\omega_l}{(\sigma_l + \varepsilon h_0^2)^{\eta_l}} \quad \eta_l = \begin{cases} 1, & \text{if } r = 1 \\ 3, & \text{if } r = 2 \\ 4, & \text{if } r \geq 3 \end{cases} \quad (9)$$

$$\bar{\omega}_l = \frac{\check{\omega}_l}{\sum_k \check{\omega}_k} \quad (10)$$

The the second stage is calculated based on first stage:

$$\hat{\omega}_l = \frac{\bar{\omega}_l}{(\sigma_l + \varepsilon h_0^2)^{sr_l}} \quad (11)$$

$$\tilde{\omega}_l = \frac{\hat{\omega}_l}{\sum_k \hat{\omega}_k} \quad (12)$$

I write the reconstruction as combination of polynomials:

$$\mathcal{R}(\mathbf{x}) = \sum_{l \in L} P_l(\mathbf{x}) \tilde{\omega}_l(\bar{u}) \quad (13)$$

Suppose the reconstruction happens in a given stencil I . The cell averaged values $\bar{u}$ in this stencil is represented as $\{\bar{u}_s\}_{s \in S}$. The derivative of this reconstruction with respect to $\bar{u}_s$ is:

$$\frac{\partial \mathcal{R}}{\partial \bar{u}_s} = \sum_{l \in L} \left(\frac{\partial P_l}{\partial \bar{u}_s} \tilde{\omega}_l + P_l \frac{\partial \tilde{\omega}_l}{\partial \bar{u}_s} \right) \quad (14)$$

It consists of two parts, derivative of polynomials $\frac{\partial P_l}{\partial \bar{u}_s}$ and derivative of non linear weights $\frac{\partial \tilde{\omega}_l}{\partial \bar{u}_s}$. The stencil polynomial is a combination of single polynomials satisfying

$$\frac{1}{|E_k|} \int_{E_k} p_s(\mathbf{x}) d\mathbf{x} = \begin{cases} 1 & s = k \\ 0 & s \neq k \end{cases} \quad (15)$$

The stencil polynomial is expressed in terms of combination of all these polynomials

$$P(\mathbf{x}) = \sum_{s \in S} p_s(\mathbf{x}) \bar{u}_s \quad (16)$$

The derivative of polynomial is trivial. The derivative of non linear weight is a little more complicated.

$$\frac{\partial \mathcal{R}}{\partial \bar{u}_s} = \sum_{l \in L} p_s(\mathbf{x}) \tilde{\omega}_l + \sum_{l \in L} P_l \frac{\partial \tilde{\omega}_l}{\partial \bar{u}_s} \quad (17)$$

3 Implicit time stepping

3.1 Backward Euler

3.2 SSP time stepping

4 Parallelism

Parallel solver is inevitable in large scale computation. Figure 1 shows a full view of quadrilateral (logically rectangular) mesh. Counting all stencils without repeating

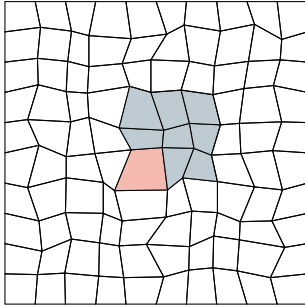


Fig. 1 Logically rectangular mesh displayed in full. The index of a stencil (colored cells) is the same as the index of the red cell.

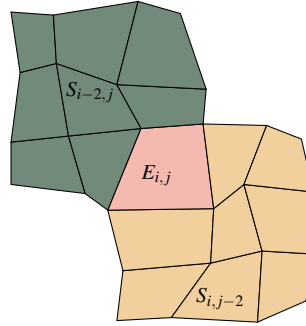


Fig. 2 Two sample stencils used by cell $E_{i,j}$. The index of these two stencils are $S_{i-2,j}$ and $S_{i,j-2}$. Consistent with the index of bottom left cell.

is convenient in the logically rectangular mesh setting. We use the index of bottom left cell as the index of the corresponding stencil. A $M \times N$ mesh will then contain $(M - I + 1) \times (N - J + 1)$ stencils of size $I \times J$.

4.1 Mesh partition

I examine the following partition of mesh without lossing generality. The dashed line indicates ghost cells distributed by MPI communication. The size of ghost layer is determined by the largest stencil size required for cells in the ghost layer. Increasing the order of reconstruction (increasing the size of corresponding stencil) will thus increase the communication cost. However, the distribution of mesh only occur once in the beginning of the computation. I will not worry about it considering we are running on the 2D mesh right now.

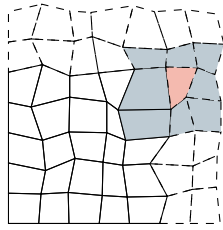


Fig. 3 Illustration of mesh partition. Ghost cells are drawn with dashed lines. The size of ghost layer is determined by the size of stencils needed by cell in the ghost layer.

4.2 Sparsity pattern of Jacobian matrix

4.3 Optimal preconditioner

- 1. Incomplete LU decomposition
- 2. Additive schwarz

4.4 Scalability